

Hanoi University of Science and Technology
School of Information and Communications Technology



PROJECT REPORT

SUBJECT: OPERATING SYSTEM - IT3070E

READER - WRITER PROBLEM

Student implementation:	Vũ Đỗ Minh Thảo 202416752 Nguyễn Khánh Linh 202416805 Đặng Thuỳ Minh 202416808 Vũ Đức Duy 202416686
Supervisor teacher:	PhD. Đỗ Quốc Huy
Class number:	166238

Hanoi, June 2026



TABLE OF CONTENTS

TABLE OF CONTENTS.....	1
CHAPTER 1. INTRODUCTION.....	2
1.1 Problem Statement.....	2
1.2 Objectives.....	2
1.3 Overview.....	3
CHAPTER 2. METHODOLOGY.....	4
2.1 Readers' Priority.....	5
2.1.1 Theory.....	5
2.1.2 Application.....	5
2.1.3 Results.....	7
2.2 Writers' Priority.....	9
2.2.1 Theory.....	9
2.2.2 Application.....	9
2.2.3 Results.....	12
2.3 Fairness Solution.....	14
2.3.1 Theory.....	14
2.3.2 Application.....	14
2.3.3 Results.....	17
CHAPTER 3: RESULTS AND COMPARISONS.....	18
3.1 Experimental Environment.....	18
3.2 Simulation Results and Comparisons.....	19
CHAPTER 4: CONCLUSION.....	20

CHAPTER 1. INTRODUCTION

1.1 Problem Statement

In concurrent computing systems, especially during the widespread use of multi-core processors and distributed cloud networks, multiple processes or threads frequently require access to the same shared resource. Without proper synchronization, this can quickly lead to data corruption and inconsistent results. The Reader-Writer Problem addresses this by defining strict access rules for two types of operations:

- **Writers:** Require strict mutual exclusion. Only one writer can modify the data at a time, and no readers can be active during a write operation to ensure data correctness.
- **Readers:** Require concurrent access. Multiple readers can safely access the data simultaneously as long as no writer is modifying it for efficiency.

The primary challenge is designing a mechanism that balances the need for mutual exclusion and concurrency while preventing starvation - a scenario where either continuous readers or writers block the other indefinitely.

1.2 Objectives

The primary objective of this project is to implement and analyze three solutions for the Reader-Writer Problem: reader-priority, writer-priority, and fair scheduling, using an interactive desktop simulator using Python and Tkinter. Specifically, the project aims to:

- Understand key concepts of different approaches.
- Utilize semaphores, mutexes, and locks to coordinate access between readers and writers.
- Guarantee data consistency, avoid starvation, race condition, and visualize thread behavior in real-time.

1.3 Overview

The remainder of this report is organized as follows to provide a comprehensive analysis and demonstration of the Reader-Writer Problem:

- **Chapter 2: Methodology and Implementation** details the theoretical algorithms and practical development of the three synchronization approaches: Readers' Priority, Writers' Priority, and the Fairness Solution. It explains the synchronization logic using semaphores and mutexes, along with how these concepts are translated into a Python application.
- **Chapter 3: Results and Comparisons** presents the outcomes of the Tkinter-based simulation. It presents the experimental environment, analyzes the behavioral characteristics and starvation properties of each approach, and provides a comprehensive comparison of the three implemented algorithms.
- **Chapter 4: Conclusion** summarizes the key findings of the project, evaluates the effectiveness of the solutions, and concludes the report.

CHAPTER 2. METHODOLOGY

In this project, the Python threading library serves as the bridge between the theoretical Reader-Writer algorithms and actual, executable concurrency by enabling concurrent execution and providing synchronization tools.

Standard Python scripts execute sequentially. However, the Reader-Writer problem requires multiple entities attempting to act simultaneously. The threading library allows multiple lightweight sub-processes (threads) that share the same memory space to spawn. Representing each Reader and Writer as a distinct threading. Thread forces the operating system to juggle its execution, perfectly simulating the unpredictable arrival times and context-switching that cause race conditions - the state where two or more threads trying to access a data at the same time and leads to the unpredictable state of the data afterwards - in the real world.

Additionally, the threading library provides essential synchronization tools to enforce the Reader-Writer rules. The `threading.Lock()` is a mutual exclusion object that guarantees operations like `readcount += 1` are executed atomically to prevent data corruption. Meanwhile, `threading.Semaphore()` provides an advanced signaling mechanism; when initialized as a binary semaphore (`Semaphore(1)`), it serves as the primary access_token for the shared resource, allowing coordinated access.

Beyond the algorithm-level primitives, a dedicated `SimulationState` class employs its own `threading.Lock()` to protect shared UI state - including the pending, reading, and writing thread pools, independently from each algorithm's synchronization logic, ensuring race-condition-free real-time updates.

To realistically simulate concurrent arrivals, each thread is subject to a randomized arrival delay (`random.uniform(0.1, 1.5)s`) and task duration (`random.uniform(1.0, 3.0)s`), with the initial arrival order further shuffled by the controller. This intentional non-determinism mirrors real-world OS scheduling behavior.

2.1 Readers' Priority

2.1.1 Theory

The Reader Priority strategy gives strict precedence to readers, ensuring that read operations are processed as quickly and continuously as possible. Specifically, when a reader thread arrives and requests access, the system immediately allows it to enter the critical section, provided that at least one other reader is currently using the shared resource. Even if one or more writers are already queued and waiting to edit the data, newly arriving readers are permitted to bypass them and join the active readers. A writer is only granted access to the resource once all readers have finished and the critical section is empty.

This approach guarantees maximum efficiency for read operations, as readers are never kept waiting if the resource is open for reading. However, when readers arrive continuously, writers will experience starvation because they are blocked indefinitely until the reader queue is empty.

Therefore, the Reader Priority strategy is a trade-off between high read efficiency and possibility of writer starvation. It is particularly suitable for systems where data updates are infrequent, read operations demand is high, and delaying writers is an acceptable compromise.

2.1.2 Application

```
import threading
from algorithms.base import AccessPolicy

class ReaderPriority(AccessPolicy):
    def __init__(self, runtime_state):
        super().__init__(runtime_state)
        self.reader_total = 0 # number of active readers
        self.counter_guard = threading.Lock()
        self.access_token = threading.Semaphore(1) # shared resource lock
```

Figure 2.1.2.1. ReaderPriority initialization code

In this ReaderPriority class, the application relies on three variables:

- `self.reader_total`: an integer variable tracking the number of readers currently in the critical section.
- `self.counter_guard`: a `threading.Lock()` used to enforce mutual exclusion over the `reader_total` variable. This prevents race condition.
- `self.access_token`: a binary `threading.Semaphore(1)` that controls access to

the critical section. It guarantees that if a writer holds the token, no one else can enter, and if readers hold the token, no writer can enter.

```
def acquire_read(self, thread_name):
    self.runtime_state.add_waiting(thread_name)
    with self.counter_guard:
        self.reader_total += 1
        if self.reader_total == 1: # first reader blocks writers
            self.access_token.acquire()
    self.runtime_state.remove_waiting(thread_name)
    self.runtime_state.add_reader(thread_name)
    self.runtime_state.append_log(f"{thread_name} started reading")

def release_read(self, thread_name):
    with self.counter_guard:
        self.reader_total -= 1
        if self.reader_total == 0: # last reader allows writers to proceed
            self.access_token.release()
    self.runtime_state.remove_reader(thread_name)
    self.runtime_state.task_completed()
    self.runtime_state.append_log(f"{thread_name} finished reading")
```

Figure 2.1.2.2. Reader logic for ReaderPriority class

The logic is handled in the `acquire_read` and `release_read` methods. When a thread attempts to read, it must first acquire the `counter_guard` lock. Inside this protected block, it increments `reader_total`. If the arriving reader is the first one (`self.reader_total == 1`), it will acquire `access_token`. Else, if a writer is active, this first reader will block until the writer finishes.

Because the first reader already had `access_token`, subsequent readers bypass the `access_token.acquire()` step, increment the counter and enter the critical section immediately, which is the core of readers' priority.

When readers finish, they decrement the counter inside the `release_read` lock. The very last reader to leave the critical section (`self.reader_total == 0`) will execute `self.access_token.release()`, unlocking the resource for any waiting writers.

```
def acquire_write(self, thread_name):
    self.runtime_state.add_waiting(thread_name)
    self.access_token.acquire() # writer must wait until all readers leave
    self.runtime_state.remove_waiting(thread_name)
    self.runtime_state.add_writer(thread_name)
    self.runtime_state.append_log(f"{thread_name} started writing")

def release_write(self, thread_name):
    self.runtime_state.remove_writer(thread_name)
    self.access_token.release() # release resource for next reader
    self.runtime_state.task_completed()
    self.runtime_state.append_log(f"{thread_name} finished writing")
```

Figure 2.1.2.3. Writer logic for ReaderPriority class

The writer logic in this approach is intentionally simple and subordinate to the readers. In `acquire_write`, the writer simply attempts to acquire the `access_token`. If there are zero readers and zero other writers, it acquires the token immediately and proceeds. If even a single reader is currently in the system, the writer is put to sleep. Because subsequent readers do not need to acquire the token, they can stream in endlessly, which illustrates writer starvation.

2.1.3 Results

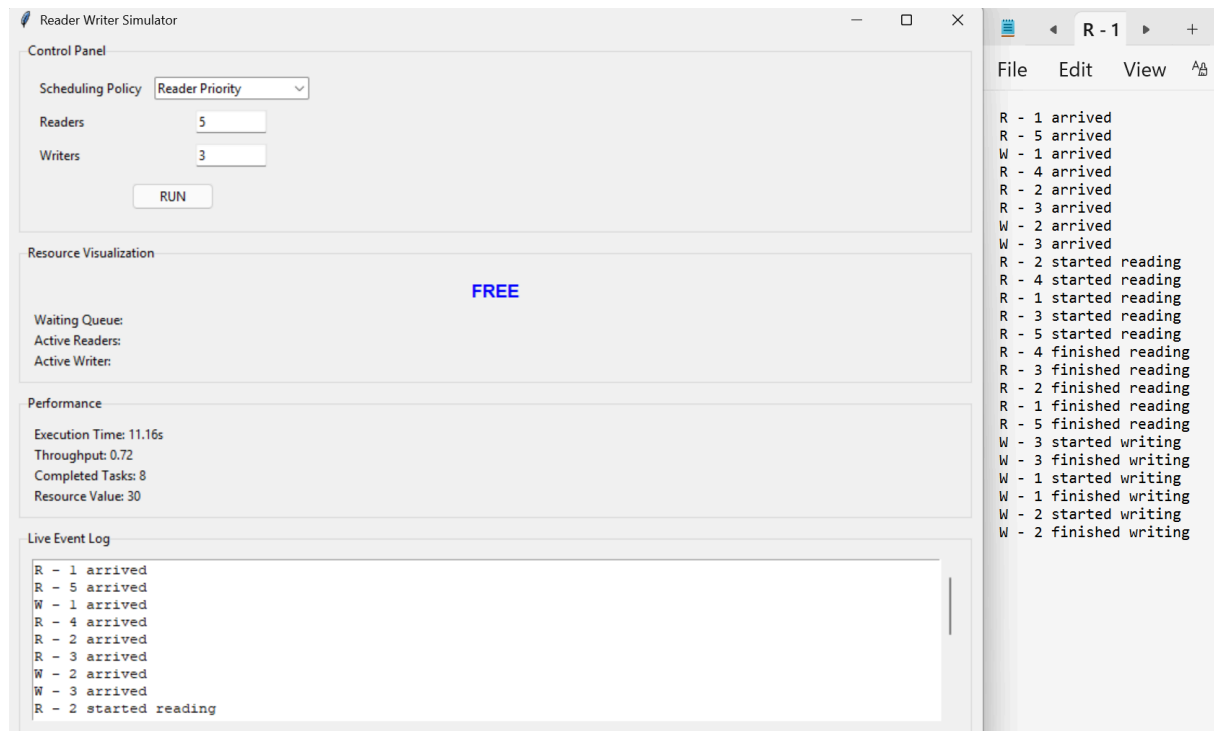


Figure 2.1.3. Reader Priority program output

The simulation log validates the Reader-Priority algorithm while demonstrating both writer starvation and unpredictable thread scheduling. Readers R-1 and R-5 arrive first and lock the system open for reading. Because of the reader-priority rules, later-arriving readers (R-4, R-2, R-3) are allowed to skip the line and enter the critical section concurrently. Meanwhile, writers W-1, W-2, and W-3 are left outside until every single reader finishes. When the final reader are finally done, exits the system and drops the lock, the operating system's random thread scheduler takes over. Writer W-3, despite arriving last, snatches the lock first and executes ahead of W-1 and W-2, proving that priorities do not guarantee a First-In-First-Out queue.

2.2 Writers' Priority

2.2.1 Theory

The Writer Priority approach is a synchronization strategy designed to address the Reader-Writer problem while preventing writer starvation. In traditional Reader Priority systems, newly arriving readers may continuously gain access to the shared resource, causing waiting writers to wait indefinitely. To tackle this limitation, the Writer Priority strategy gives precedence to writers whenever one or more writers are waiting for access.

When a writer thread arrives and requests access, the system prevents any new reader threads from entering the critical section. Existing readers that are already accessing the shared resource are allowed to complete their operations normally. Once all active readers have finished, the waiting writer is granted exclusive access to the resource.

This approach guarantees that writers will eventually obtain the resource regardless of the number of newly arriving readers. As a result, writer starvation is eliminated and write operations can be performed in a timely manner. However, prioritizing writers may increase reader waiting times. In situations where writers arrive continuously, readers may experience starvation because they must wait until all pending writers have completed their operations.

Therefore, the Writer Priority strategy represents a trade-off between fairness and responsiveness to write requests. It is particularly suitable for systems where write operations are critical and should not be delayed for extended periods.

2.2.2 Application

In this `WriterPriority` class, the application relies on five synchronization variables:

```
class WriterPriority(AccessPolicy):
    def __init__(self, runtime_state):
        super().__init__(runtime_state)
        self.active_reader_total = 0
        self.queued_writer_total = 0
        self.reader_counter_guard = threading.Lock()
        self.writer_counter_guard = threading.Lock()
        self.entry_gate = threading.Semaphore(
            1
        ) # Blocks new readers when writers are waiting
        self.resource_token = threading.Semaphore(1) # Shared resource lock
```

Figure 2.2.2.1 WriterPriority initialization code

- `Self.active_reader_total`: variable that tracks the number of readers currently accessing the shared resource.
- `self.queued_writer_total`: variable that records the number of writers waiting to perform write operations.
- `self.reader_counter_guard`: a `threading.Lock()` that protects updates to `active_reader_total`, ensuring that reader count modifications are performed atomically.
- `self.writer_counter_guard`: a `threading.Lock()` that protects updates to `queued_writer_total`, preventing race conditions when multiple writers arrive simultaneously.
- `self.entry_gate`: a binary `threading.Semaphore(1)` that serves as a gate controlling whether new readers are allowed to enter the system. When at least one writer is waiting, this gate is closed to block additional readers.
- `self.resource_token`: a binary `threading.Semaphore(1)` that provides exclusive access to the shared resource. It guarantees that only one writer, or a group of readers, may access the resource at any given time.

The reader logic is implemented in the `acquire_read` and `release_read` methods.

```
def acquire_read(self, thread_name):
    self.runtime_state.add_waiting(thread_name)
    # Readers must pass through the writer gate
    self.entry_gate.acquire()
    self.entry_gate.release()
    with self.reader_counter_guard:
        self.active_reader_total += 1
        if self.active_reader_total == 1: # First reader locks the resource
            self.resource_token.acquire()
    self.runtime_state.remove_waiting(thread_name)
    self.runtime_state.add_reader(thread_name)
    self.runtime_state.append_log(f"{thread_name} started reading")
```

Figure 2.2.2.2. `acquire_read` in `WriterPriority` class

Before entering the critical section, every reader must first pass through the `entry_gate`. If no writer is waiting, the reader immediately acquires and releases the gate, allowing execution to continue. However, if a writer has already closed the gate, the reader is suspended until writers have completed their operations. After passing the gate, the reader acquires `reader_counter_guard` and increments `active_reader_total`. If this reader is the first active reader (`self.active_reader_total == 1`), it acquires `resource_token`, preventing writers from accessing the shared

resource. Subsequent readers only update the reader count and immediately join the existing reading group, allowing multiple readers to execute concurrently.

```
def release_read(self, thread_name):
    with self.reader_counter_guard:
        self.active_reader_total -= 1
        if self.active_reader_total == 0: # Last reader releases the resource
            self.resource_token.release()
    self.runtime_state.remove_reader(thread_name)
    self.runtime_state.task_completed()
    self.runtime_state.append_log(f"{thread_name} finished reading")
```

Figure 2.2.2.3. release_read in WriterPriority class

When a reader finishes, it acquires reader_counter_guard again and decrements active_reader_total. If it is the last active reader (self.active_reader_total == 0), it releases resource_token, allowing a waiting writer to obtain exclusive access.

The writer logic is implemented in the acquire_write and release_write methods.

```
def acquire_write(self, thread_name):
    self.runtime_state.add_waiting(thread_name)
    with self.writer_counter_guard:
        self.queued_writer_total += 1
        if self.queued_writer_total == 1: # First waiting writer blocks new readers
            self.entry_gate.acquire()
    # Writer requires exclusive access
    self.resource_token.acquire()
    self.runtime_state.remove_waiting(thread_name)
    self.runtime_state.add_writer(thread_name)
    self.runtime_state.append_log(f"{thread_name} started writing")
```

Figure 2.2.2.4. acquire_write in WriterPriority class

When a writer arrives, it first acquires writer_counter_guard and increments queued_writer_total. If it is the first waiting writer (self.queued_writer_total == 1), it acquires entry_gate. This immediately blocks all future readers from entering the system while allowing any readers that are already active to complete normally.

After closing the reader gate, the writer attempts to acquire resource_token. If readers are still active, the writer blocks until the last reader releases the resource. Once resource_token is obtained, the writer gains exclusive access to the shared resource and performs its write operation.

```
def release_write(self, thread_name):
    self.runtime_state.remove_writer(thread_name)
    self.resource_token.release()
    with self.writer_counter_guard:
        self.queued_writer_total -= 1
        # Reopen reader gate when no writers remain
        if self.queued_writer_total == 0:
            self.entry_gate.release()
    self.runtime_state.task_completed()
    self.runtime_state.append_log(f"{thread_name} finished writing")
```

Figure 2.2.2.5. release_write in WriterPriority class

Upon completion, the writer releases resource_token and decrements queued_writer_total inside writer_counter_guard. If no writers remain waiting (self.queued_writer_total == 0), it releases entry_gate, allowing readers to enter the system once again.

2.2.3 Results

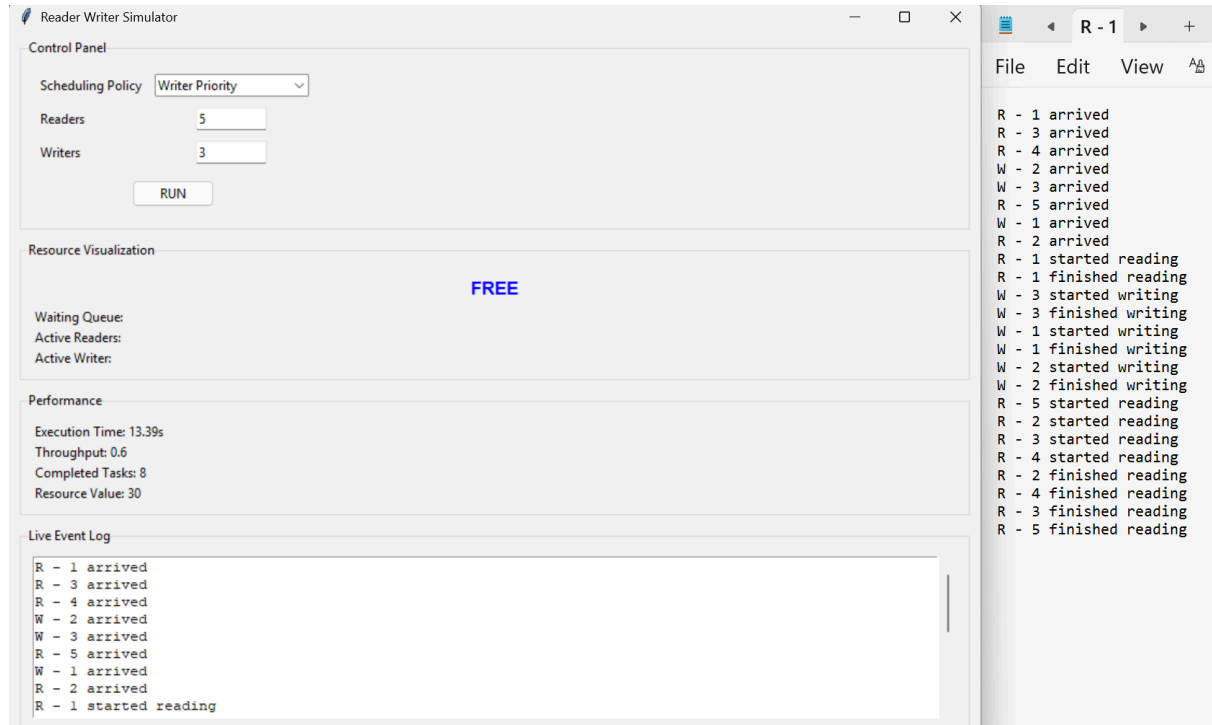


Figure 2.2.3. Writer Priority program output

As the event log shows, reader R-1 arrives first, successfully secures the resource token, and begins reading. However, while R-1 is active, writers W-2 and W-3 enter. Once R-1 completes, the system halts all reader progress and processes the

writers sequentially to guarantee mutual exclusion and prevent data corruption. Only after the very last writer W-2 finishes its execution, dropping the waiting writer count to zero, that readers are allowed. Readers R-3 and R-4 arrived before any writers, yet were forced to wait until this point.

This log perfectly illustrates both the function and the vulnerability of Writers' Priority approach - reader starvation.

2.3 Fairness Solution

2.3.1 Theory

The Fairness Solution is a synchronization strategy designed to completely eliminate starvation for both readers and writers. While the Reader Priority and Writer Priority algorithms inherently favor one class of threads at the expense of the other—often leading to indefinite waiting—the Fairness approach enforces a strict First-Come-First-Serve (FCFS) ordering mechanism.

In this system, every incoming thread, whether a reader or a writer, must pass through a common entry queue (a turnstile). Once a thread signals its intent by entering the queue, it prevents any subsequent threads from bypassing it. If a writer is queued, incoming readers must wait until the writer completes; conversely, if readers are queued, incoming writers must wait for their turn.

This approach guarantees that all processes are granted access to the shared resource in a finite amount of time, ensuring absolute fairness. However, the trade-off is a potential reduction in overall system concurrency and throughput. Multiple readers can no longer overlap their reads if a single writer arrives and queues up in between them. Therefore, this strategy is highly optimal for systems where predictable response times and strict equity among operations are more critical than maximizing raw parallel performance.

2.3.2 Application

In the `FairnessFCFS` class, the theoretical First-Come-First-Serve (FCFS) synchronization logic is translated into executable code utilizing Python's built-in threading module. The following code snippets demonstrate how the "turnstile" concept and resource locks are effectively managed to prevent starvation for both readers and writers.

```
class FairnessFCFS(AccessPolicy):
    def __init__(self, runtime_state):
        super().__init__(runtime_state)
        self.reader_group_size = 0 # Number of readers currently sharing the resource
        self.arrival_turnstile = threading.Semaphore(
            1
        ) # Arrival queue (turnstile) to preserve FCFS ordering
        self.resource_token = threading.Semaphore(
            1
        ) # Token to control access to the shared resource
        self.counter_guard = threading.Lock() # Protect reader counter updates
```

Figure 2.3.2.1. FairnessFCFS initialization code

The `arrival_turnstile` semaphore acts as a common queue that preserves First-Come-First-Serve ordering among all incoming threads. The `resource_token` semaphore protects the shared resource, while the `counter_guard` mutex ensures that updates to `reader_group_size` are performed atomically.

```
def acquire_read(self, thread_name):
    self.runtime_state.add_waiting(
        thread_name
    ) # Reader arrives and joins waiting queue
    self.arrival_turnstile.acquire() # Wait for its turn in arrival order
    with self.counter_guard:
        self.reader_group_size += 1
        if self.reader_group_size == 1: # First reader locks the shared resource
            self.resource_token.acquire()
    self.arrival_turnstile.release() # Allow next arriving request to proceed
    self.runtime_state.remove_waiting(thread_name) # Reader enters critical section
    self.runtime_state.add_reader(thread_name)
    self.runtime_state.append_log(f"{thread_name} started reading")
```

Figure 2.2.3.2. acquire_read in FairnessFCFS class

When a reader arrives, it first joins the waiting queue and acquires the `arrival_turnstile` semaphore. If another thread is currently passing through the turnstile, the underlying threading library blocks the reader instead of allowing busy waiting.

Inside the critical section protected by `counter_guard`, the reader increments `reader_group_size`. If this is the first active reader, it acquires `resource_token`, preventing writers from accessing the shared resource.

After securing access, the reader immediately releases `arrival_turnstile`, allowing the next request to proceed according to FCFS ordering.

```
def release_read(self, thread_name):
    with self.counter_guard:
        self.reader_group_size -= 1
        if self.reader_group_size == 0:
            self.resource_token.release()
    self.runtime_state.remove_reader(thread_name) # Reader leaves critical section
    self.runtime_state.task_completed()
    self.runtime_state.append_log(f"{thread_name} finished reading")
```

Figure 2.3.2.3. release_read in FairnessFCFS class

When a reader finishes, it acquires the `counter_guard` lock to safely decrement `reader_group_size`. If it is the last active reader leaving the critical section (`reader_group_size == 0`), it releases the `resource_token`. This critical step

unblocks any writer that is currently at the front of the queue waiting for the resource.

```
def acquire_write(self, thread_name):
    self.runtime_state.add_waiting(
        thread_name
    ) # Writer arrives and joins waiting queue
    self.arrival_turnstile.acquire() # Wait in FCFS order
    self.resource_token.acquire() # Writer requires exclusive ownership
    self.arrival_turnstile.release() # Allow next request to enter the arrival queue
    self.runtime_state.remove_waiting(thread_name) # Writer enters critical section
    self.runtime_state.add_writer(thread_name)
    self.runtime_state.append_log(f"{thread_name} started writing")
```

Figure 2.3.2.4. acquire_write in WriterPriority class

A writer thread follows the exact same entry protocol. It must acquire the arrival_turnstile to cement its position in the FIFO queue. Once it is its turn, it attempts to acquire the resource_token. Only after successfully securing the token does the writer release the arrival_turnstile. This ensures that while the writer is performing its operation, the next thread can move past the turnstile but will be stopped at the resource_token until the write is complete.

```
def release_write(self, thread_name):
    self.runtime_state.remove_writer(
        thread_name
    ) # Writer releases exclusive ownership
    self.resource_token.release()
    self.runtime_state.task_completed()
    self.runtime_state.append_log(f"{thread_name} finished writing")
```

Figure 2.3.2.5. release_write in WriterPriority class

After completing its write operation, the writer retrieves exclusive ownership by releasing resource_token. This enables the next waiting thread to proceed, thereby preserving FCFS ordering and guaranteeing bounded waiting for all threads.

2.3.3 Results

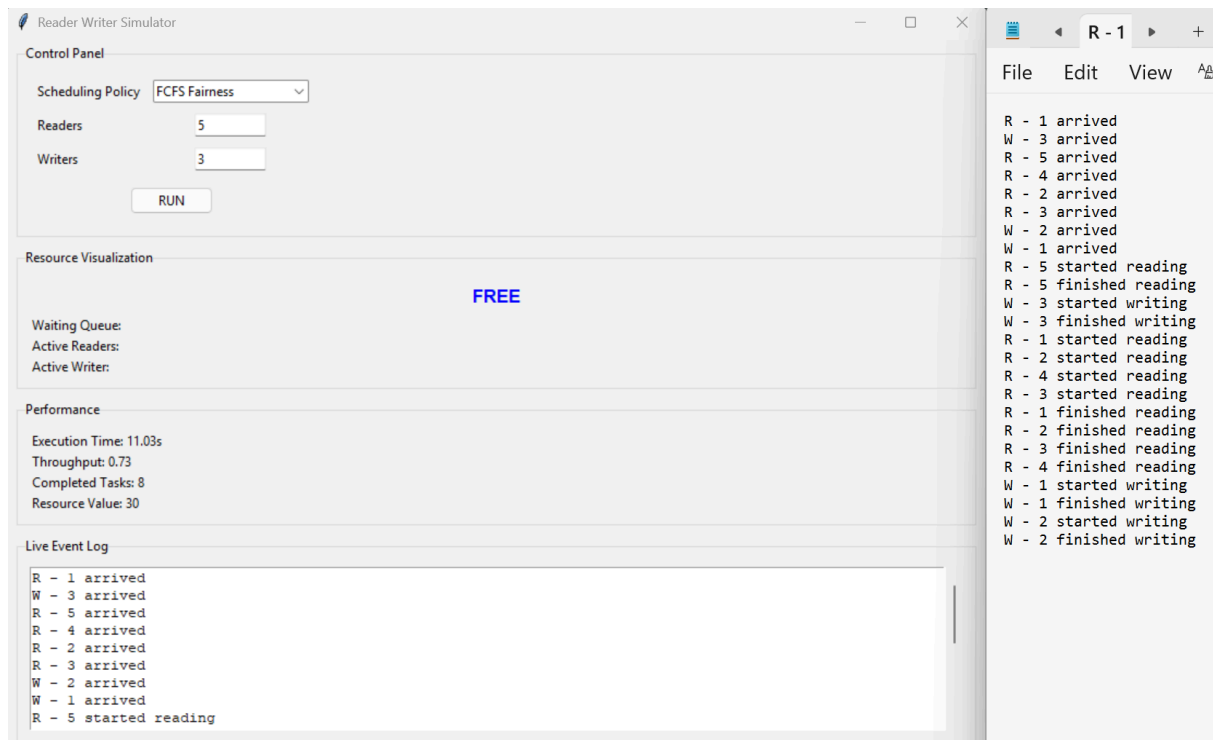


Figure 2.3.3. Fairness FCFS program output

Due to the natural unpredictability of OS thread scheduling, reader R-5 secures the access token first and enters the critical section. However, the critical difference occurs immediately after: writer W-3, which arrived early in the simulation, secures the turnstile right behind R-5, preventing starvation. Once W-3 completes and releases the resource, the remaining readers share the resource token and execute concurrently, maintaining system efficiency. Finally, the remaining queued writers (W-1 and W-2) execute. Overall, the output above validates the FCFS algorithm and the elimination of starvation.

CHAPTER 3: RESULTS AND COMPARISONS

3.1 Experimental Environment

The Reader-Writer Synchronization Simulator was implemented using Python and the Tkinter graphical user interface framework. The simulation employs Python threads to represent concurrent reader and writer processes while synchronization is achieved through locks and semaphores.

For each execution, a configurable number of reader and writer threads are created. To simulate real-world concurrent systems, thread arrival times and operation durations are randomized. Specifically, each thread experiences a random arrival delay before requesting access to the shared resource and a random execution duration while performing its read or write operation.



```

1  import random
2  import threading
3  import time
4
5
6  class ReaderThread(threading.Thread):
7
8      def __init__(self, worker_id, access_policy, shared_resource, runtime_state):
9          super().__init__()
10         self.worker_id = worker_id
11         self.access_policy = access_policy
12         self.shared_resource = shared_resource
13         self.runtime_state = runtime_state
14     def run(self):
15         self.runtime_state.append_log(f"{self.worker_id} arrived") # reader request
16         arrived
17         time.sleep(random.uniform(0.1, 1.5)) #simulated arrival delay
18         self.access_policy.acquire_read(self.worker_id) #request read permission
19         _ = self.shared_resource.read() #critical section
20
21         time.sleep(random.uniform(1.0, 3.0)) #simulated reading time
22         self.access_policy.release_read(self.worker_id) # release resource

```

Figure 3.1.1. Readers threads' time variable randomized (same for writers)

Because thread scheduling is ultimately controlled by the operating system, the execution order and time may vary between simulation runs. Therefore, the evaluation focuses primarily on synchronization behavior, fairness properties, and starvation characteristics rather than comparison.

3.2 Simulation Results and Comparisons

The simulator records the execution sequence of reader and writer threads and displays the current system state, including waiting queue active readers, active writers, and resource status.

Although execution times may vary between runs, the three synchronization strategies consistently exhibit distinct behavioral characteristics.

Table 3.2.1. Properties comparison of three synchronization strategies

Property	Reader priority	Writer priority	Fairness FCFS
Reader preference	High	Low	Balanced
Writer Preference	Low	High	Balanced
Reader Starvation	No	Possible	No
Writer Starvation	Possible	No	No
Fairness	Low	Low	High
Resource Sharing among readers	Yes	Yes	Yes
Exclusive writer access	Yes	Yes	Yes

Reader Priority favors read-intensive workloads by allowing multiple readers to access the resource as long as there is at least one reader current reading. However, continuous reader arrivals may result in writer starvation. Writer Priority addresses this limitation by granting precedence to writers whenever one or more writers are waiting. While this eliminates writer starvation, reader starvation is possible. Fairness FCFS seeks to balance both by preserving request order - the first arrived will be executed first. This prevents starvation for both parties and provides the most reasonable resource allocation among the three algorithms.

CHAPTER 4: CONCLUSION

This project successfully implemented and visualized the Reader-Writer synchronization problem using Python threads, semaphores, and locks. Three synchronization strategies were developed and evaluated: Reader Priority, Writer Priority, and Fairness FCFS.

The simulation results demonstrate the trade-offs associated with each approach. Reader Priority provides efficient access for readers but may lead to writer starvation. Writer Priority eliminates writer starvation by prioritizing waiting writers, although reader starvation is possible as a result. Fairness FCFS achieves balanced resource allocation and prevents starvation for both readers and writers.

In addition to demonstrating synchronization concepts, the graphical simulation provides a clear visualization of thread interactions, resource allocation, waiting queues, and system state transitions. This makes the application a useful educational tool for understanding concurrent programming and operating system synchronization mechanisms.

Future improvements may include collecting additional performance metrics such as average waiting time, resource utilization, and starvation frequency. More advanced synchronization strategies and larger-scale workload simulations may also be incorporated to further understand system behavior under different operating conditions.